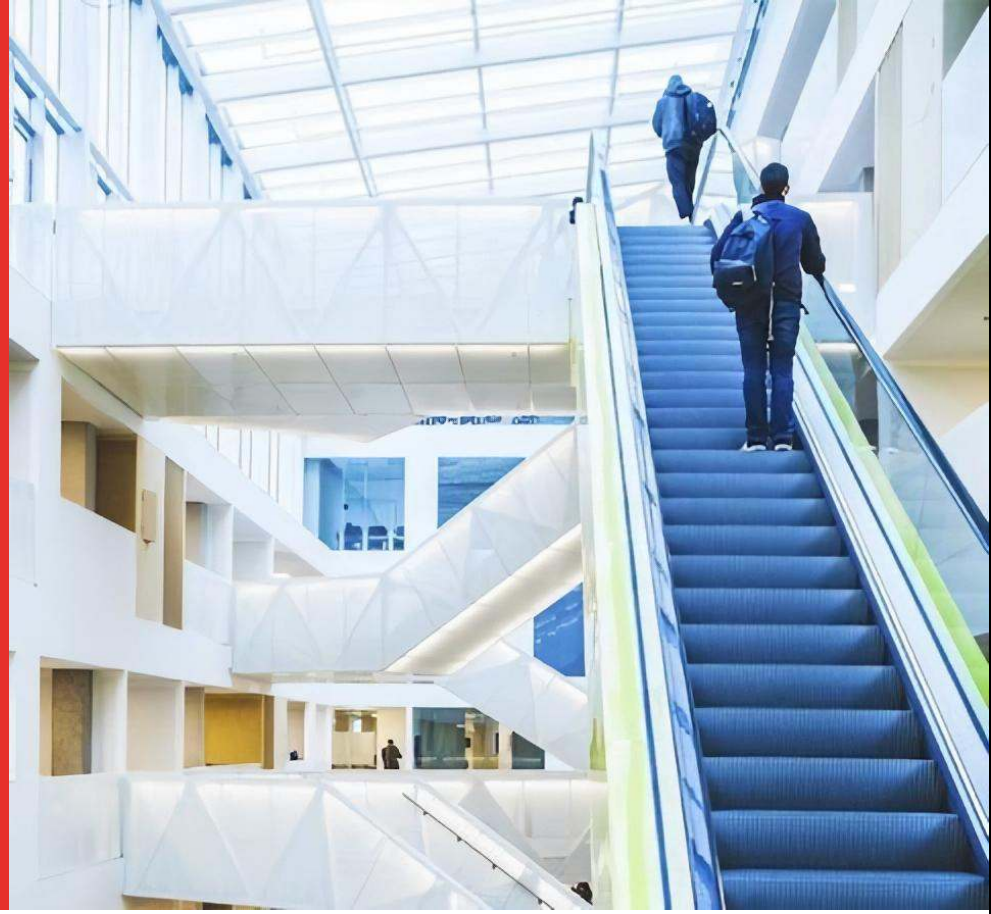
DATASTRUCTUREN



HOGESCHOOL UTRECHT

HIER KOMT ALLES SAMEN

LEERDOELEN

Na deze les begrijp en pas je toe:

- Big O notatie en hoe je algoritmes analyseert op efficiëntie
- lineaire en associatieve datastructuren
- wanneer je welke structuur gebruikt op basis van operatiekosten
- hoe data*structuren* Big O karakteristieken bepalen

PROGRAMMA

1. Big-O recap
2. Datastructuren
3. Lineaire datastructuren
4. Associative datastructuren

BIG-O RECAP

```python
# Algoritme A:
def find_max(arr):
    return max(arr)
```

- Algoritme A: lineair, schaalbaar

```python
# Algoritme B:
def find_max(arr):
    for item in arr:
        for other in arr:
            if other > item:
                break
        else:
            return item
```

- Algoritme B: hoe groter de array, hoe langzamer

BIG O NOTATIE: KERNIDEE

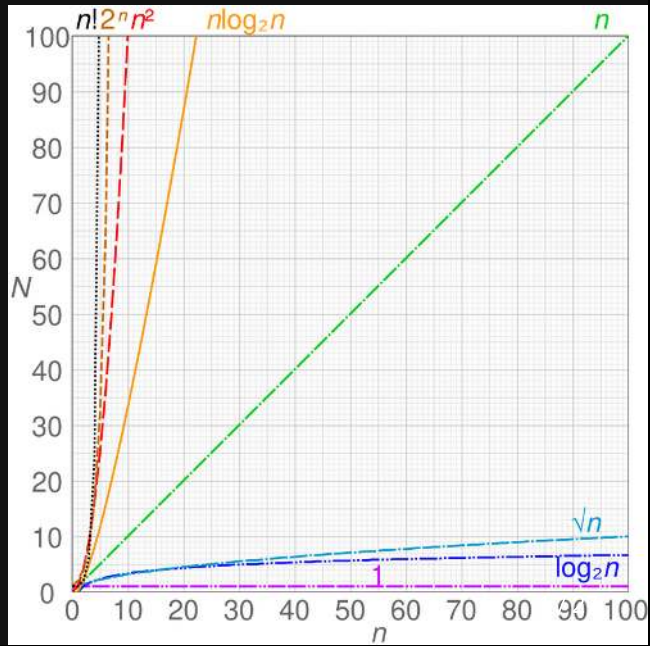
Big O beschrijft hoe runtime groeit als de input groeit:

$$O(f(n)) \text{ betekent: 'groeit als'} \approx f(n)$$

De meest voorkomende ordes:

- O(1): Constante tijd
- O(log n): Logaritmische tijd
- O(n): Lineaire tijd
- O(n log n): Linearitmische tijd
- O(n^2): Kwadratische tijd
- O(2^n): Exponentiële tijd
- O(n!): Faculteit tijd

Waar letten we op bij Big O?



ANALYSE STAP-VOOR-STAP

Hoe bepaal je Big O?

1. Identificeer loops en herhalingen
2. Count iteraties in termen van n
3. Vermenigvuldig geneste loops
4. Negeer constanten en lagere-orde termen

Voorbeeld:

```
def example(arr):
    for i in range(len(arr)):        # n iteraties
        print(arr[i])                # → O(n)

    for i in range(len(arr)):        # n
        for j in range(len(arr)):    # n (genest)
            print(arr[i], arr[j])    # → O(n²)
```

Regel: $O(n^2 + n)$ vereenvoudigen naar $O(n^2)$

HOE ZIT HET DAN HIER MEE?

Hoe bepaal je de Big O van deze functie?

- Is lastig, want je weet de implementatie van `max` niet
- Kunt wel *aannames* maken

```python
def find_max(arr):
    return max(arr)

def max(arr):
    max = arr[0]
    for item in arr[1:]
        max = item if (item > max) else max
```

VOORBEELDEN: HOE HERKEN JE ZE

$O(1)$ — Constant:

```python
def get_first(arr):
    return arr[0]
```

$O(\log n)$ — Binary search:

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left ≤ right:
        mid = (left + right) // 2  # halveer zoekruimte
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

VOORBEELDEN (VERVOLG)

$O(n)$ — Lineair:

```python
def find_max(arr):
    max_val = arr[0]
    for item in arr:
        if item > max_val:
            max_val = item
    return max_val
```

$O(n^2)$ — Nested lussen:

```python
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(len(arr) - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

PRAKTISCHE SCHAAL: $n = 10.000$

Orde	Stappen (schatting)	Snelheid
$O(1)$	1	instant
$O(\log n)$	~13	instant
$O(n)$	10.000	instant
$O(n \log n)$	130.000	instant
$O(n^2)$	100M	~1 sec
$O(2^n)$	oneindig	onbruikbaar

→ Kleine complexiteitsverschil = gigantische impact!

CHECK-IN: BEPAAL DE COMPLEXITEIT

1. Single loop: `for i in range(n): print(i)` → ?
2. Nested loops: `for i in range(n): for j in range(n): ...` → ?
3. Binary search: halveer zoekruimte elk iteratie → ?
4. Recursief fib: twee oproepen per stap → ?

DATASTRUCTUREN

- Opslaan van data in je geheugen
- Hóe je data opslaat is belangrijk voor efficiëntie
- We kiezen op basis van *operatiekosten*
- Bovenstaande is meestal insert, delete, lookup

We maken onderscheid tussen:

- lineaire,
- associatieve,
- hiërarchische datastructuren

LINEAIRE DATASTRUCTUREN

Datastructuren waarin:

- Data sequentieel wordt opgeslagen
- Data een volgorde heeft (positie gebaseerd)
- Er een link tussen voorgaande en volgende elementen bestaat

Voorbeelden: arrays, linked list, stack, queue

ARRAYS VS LISTS: KERNIDEE

Array (conceptueel):

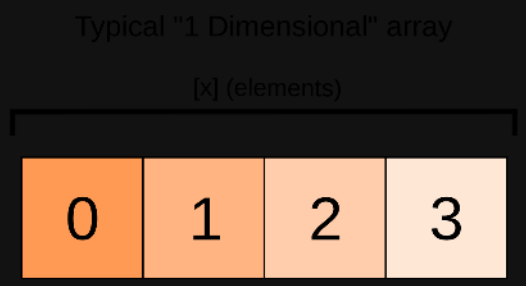
- aaneengesloten geheugen
- random access container

- wat kost insert/delete midden?
- wat kost het opzoeken van een willekeurig element?

List (implementatie, dynamische array, zoals Python list / std::vector):

- groeit en krimpt automatisch, i.t.t. array in C++

- wanneer is een dergelijke datastructuur duur?



Element indexes are typically defined in the format *[x]*
[x] being the number of elements
For example: this array could be defined as *array[4]*

STACK (LIFO)

Stack = Last In, First Out; letterlijk een stapel.

- operaties: `push`, `pop`, `peek`
- allemaal typisch $O(1)$

```
push A, push B, push C
pop → C
```

QUEUE (FIFO)

Queue = First In, First Out.

- enqueue achteraan
- dequeue vooraan
- typisch allebei $O(1)$ (met juiste implementatie)

```python
from collections import deque

q = deque(["taak1", "taak2"])
print(q.popleft())  # taak1
```

STACK EN QUEUE IN C++

```cpp
#include <stack>
#include <queue>

std::stack<int> s;
s.push(10);
s.push(20);
s.pop(); // 20 eruit

std::queue<int> q;
q.push(10);
q.push(20);
q.pop(); // 10 eruit
```

KERNVERSCHIL

- stack: laatste element eerst eruit
- queue: eerste element eerst eruit
- zelfde data, ander toegangsbeleid

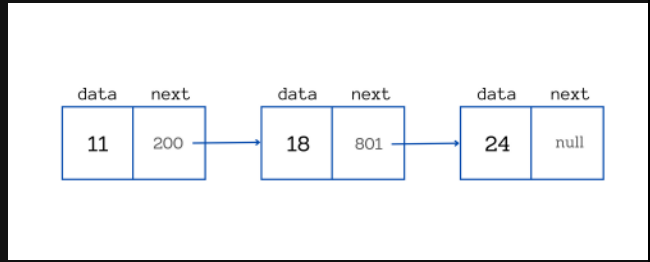
LINKED LISTS

Linked list = nodes met pointers naar de volgende node.

- geen aaneengesloten geheugen nodig
- insert/delete op bekende positie: $O(1)$
- random access: $O(n)$

Verschillende soorten:

- singly linked
- doubly linked



LINKED LISTS

```
[data|next] → [data|next] → [data|next] → null
```

Bij doubly linked:

```
null ← [prev|data|next] ↔ [prev|data|next] → n
```

WANNEER WEL/NIET?

- wel: veel inserts/deletes op bekende nodes
- niet: vaak indexeren op positie i
- cache-locality vaak slechter dan arrays/vectors

Daarna:

ASSOCIATIEVE DATASTRUCTUREN

ASSOCIATIVE DATASTRUCTUREN

Nu gaan we van positie-gebaseerd naar sleutel-gebaseerd denken:

- sets voor unieke waarden
- dictionaries/hashmaps voor key-value

SETS

Een set bewaart unieke waarden.

- geen duplicaten
- handig voor deduplicatie en snelle lookups

- wat maakt dat het hier goed voor is?

```
seen = set(["a", "b", "a"])
print(seen)       # {'a', 'b'}
print("b" in seen) # True, gemiddeld O(1)
```

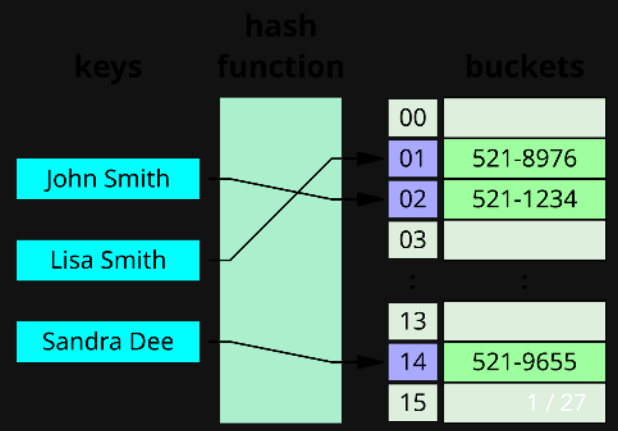
DICTIONARIES EN HASHMAPS

Een dictionary is een key-value datastructuur.

- conceptueel: mapping van sleutel naar waarde
- implementatie vaak: hashmap
- insert, lookup, delete gemiddeld $O(1)$

Voorbeelden:

- Python; `dict`
- C++; `std::unordered_map`



HASHING: INTUITIES

```
key —hash()⟶ index in bucket-array
```

Collisions bestaan:

- twee keys krijgen dezelfde index
- oplossing via chaining of open addressing

PRAKTISCHE GEVOLGEN

- gemiddeld razendsnel
- worst-case lookup kan $O(n)$ zijn
- goede hashfunctie en resizing zijn cruciaal

VERGELIJKINGSMATRIX

Structuur	Lookup	Insert	Delete	Opmerking
Array	$O(1)$	$O(n)$	$O(n)$	vaste grootte
Dynamische list/vector	$O(1)$	$O(1)$	$O(1)$	resize kost soms $O(n)$
Stack (LIFO)	$O(1)$	$O(1)$	$O(1)$	laatste element eerst eruit
Queue (FIFO)	$O(1)$	$O(1)$	$O(1)$	eerste element eerst eruit
Linked list	$O(n)$	$O(1)$*	$O(1)$*	*als node bekend is
Set (hash)	avg $O(1)$	avg $O(1)$	avg $O(1)$	unieke waarden
Dict/Hashmap	avg $O(1)$	avg $O(1)$	avg $O(1)$	key-value

OEFENVRAGEN

1. Je wilt unieke student-ID's bijhouden en snel checken of een ID al bestaat. Welke structuur?
2. Je bouwt een printqueue. Welke structuur past het best?
3. Waarom kan een slechte hashfunctie performance van een hashmap breken?
4. Je wilt steeds de hoogste prioriteitstaak eerst uitvoeren. Welke structuur kies je?

AFSLUITING

Kies een datastructuur op basis van:

- toegangs-patroon (random access, FIFO, LIFO, key-lookup)
- update-patroon (veel inserts/deletes?)
- behoefte aan ordering, uniqueness of prioriteit
- geheugen en implementatiecomplexiteit

Volgende stap: deze structuren toepassen in zoek- en sorteerproblemen.

Volgende les: hiërarchische datastructuren, grafen